

LINGXSEH Electromagnetic Field Computer: Theoretical Demonstration & Implementation Framework

中文: LINGXSEH 灵瞬鸿电磁场计算机-理论论证与实现框架

Trusted Timestamp: TSA-01-20260329236611128

Author: Jiang Wenjia (Shenzhen, China)

ORCID: 0009-0000-3850-7286

Correspondence: jiangwenjiaszx@outlook.com

Abstract

- Traditional silicon-based computing is approaching performance limits due to physical constraints: electron migration, thermal dissipation, Von Neumann bottleneck
- Propose a novel computing paradigm rooted in discrete mathematics, with direct electromagnetic field evolution as the computing carrier, based on CDEQ framework
- Demonstrate the dual nature of electromagnetic fields: energy transfer and computing are two sides of the same coin
- Realize lossless mapping from continuous electromagnetic fields to discrete computing via Q14 fixed-point quantization and three CDEQ core theorems
- Prove natural evolution of electromagnetic fields can be directly mapped to deterministic computing under conservation laws and rigid constraints
- Elaborate mathematical foundation, rigid design principles, rigorous proofs of three core theorems, and minimum viable verification framework
- Verify low-cost, normal temperature and pressure implementation feasibility; provide a new physical path to break through traditional computing bottlenecks
- Demonstrate a complete closed-loop of CDEQ framework from mathematical axioms to engineering implementation

Keywords: CDEQ Framework; Electromagnetic Field Computing; Q14 Fixed-Point; Poynting Vector; Noether's Theorem; Deterministic Convergence

1. Problem Statement: Why Can't Electromagnetic Fields Be Used for Computing?

- Established applications prove electromagnetic fields can be precisely controlled to perform deterministic work: medical electromagnetic lithotripsy, induction cookers, wireless charging
- Academic skepticism towards electromagnetic field computing stems from three core misconceptions:
 1. Carrier Bias: Recognize only electron migration as the sole physical basis of computing, ignoring information-carrying capacity of the field itself
 2. Discretization Barrier: Misbelieve continuous physical quantities cannot be directly mapped to discrete states without feasible quantization methods

3. Stability Concern: Fear uncontrollable, divergent electromagnetic field evolution, unable to deliver deterministic results like digital circuits

- Core Proposition: Electromagnetic fields are inherently capable of both energy transfer and computing, sharing the same underlying physical principles

2. Overview of CDEQ Framework

- **Full Name:** Cross-Domain-Cyber-Dynamic-Equilibrium-Regulation-Framework; a novel engineering cybernetics paradigm rooted entirely in discrete mathematics

- Core Methodology: Retrograde construction from effect to cause; anchor quantifiable target steady state, reverse derive dynamic generation mechanism, structural constraints, and convergence verification criteria

- Numerical Basis: Exclusive use of Q14 fixed-point integer arithmetic; abandon floating-point approximation and probabilistic sampling

- Three Core Theorems (Closed-Loop Control Logic):

1. PCTT-CW (Penta-Cyclic-Clockwise-Topological-Theorem): Deliver ordered, bounded, reproducible endogenous evolutionary dynamics.

2. CENHE-LEX (Central-Holding-Nonary-Constraint-Execution-Theorem): Achieve high-dimensional state space compression and structural self-stabilization via nine-grid rigid structure.

3. GCVT-10 (Global-Convergence-Validation-Theorem): Implement finite-step convergence verification with 10 consecutive cost function below threshold, replacing traditional asymptotic limits.

- Cross-domain applicability: Verified in audio synthesis, generative art; extensible to electromagnetic field computing

- All theorems: Rigorously proven via discrete mathematics, graph theory, combinatorics, number theory; judicially immutable via TSA trusted timestamp

3. Theoretical Foundation of Electromagnetic Field Computing

3.1 Unity of Electromagnetic Field Work and Computing

- Shared physical elements for energy transfer and computing:

1. Controllable Source: Precise modulation of field intensity/distribution via current, voltage

2. Rigorous Boundary: Conductor, magnetic media prevent field divergence

3. Observable State: High-precision sampling of electric field intensity, magnetic field intensity, Poynting vector

- Core Logic: Map electromagnetic field evolution to discrete logic, realize computing steps via energy flow

3.2 Physical Essence of Electromagnetic Oscillation

- Periodic conversion between electric field energy and magnetic field energy, analogous to mechanical pendulum oscillation

- Stable oscillation frequency defines deterministic computing steps; natural frequency locked by resonant cavity (e.g., microwave oven cavity)

- No external crystal oscillator required; microsecond-level stable rhythm guaranteed

3.3 Poynting Vector ($S = E \times H$)

- Defines three core computing parameters:

1. Direction: Determines topology of computing path
 2. Magnitude: Determines energy injection rate and computing step driving force
 3. Propagation Speed: Defines physical limit of single operation (speed of light)
- Precise modulation via PWM (pulse width modulation) enables programmable discrete computing

3.4 Mathematical Foundation of Three CDEQ Theorems

- **PCTT-CW Theorem:** Based on causality, Q14 fixed-point, 167ms time lattice, 1-regular directed strongly connected topology, second-order dynamics, golden ratio coefficient; prove unique evolutionary form of five-element ring

- **CENHE-LEX Theorem:** Based on eight premises including Luo Shu magic square structure; realize non-perturbative exponential convergence and robust anti-disturbance via five-anchor binding

- **GCVT-10 Theorem:** Based on four axioms; prove finite 10-step deterministic convergence via mathematical induction and reductio ad absurdum

- All proofs: TSA timestamp solidification, with complete judicial probative force

3.5 Retrograde Construction Principle (From Target to Design)

1. Anchor Target Steady State: Define five-element and nine-grid balance state as global zero-deviation point
2. Construct Deviation Metric Function: Quantify system deviation from target steady state
3. Reverse Derive Core Conditions: Dynamic generation (PCTT-CW), structural constraint (CENHE-LEX), convergence verification (GCVT-10)
4. Engineering Transformation: Convert theoretical conditions to implementable specifications (Q14 arithmetic, hard clipping, hierarchical control)

3.6 Rigid Design Principles

- Native Discreteness: Abandon continuous mathematical modeling, adopt discrete state variables and operations
- Q14 Fixed-Point Rigor: Unified benchmark value 16384 (1.0), hard clipping within [0, 16383]
- Historical Causality Drive: Evolution based solely on solidified historical state snapshots
- Structural Constraint Dominance: Stability guaranteed by innate rigid structure, not parameter tuning
- Finite-Step Convergence: Reject asymptotic convergence, adopt 10-step verification criterion
- Time-Scale Separation: Fast computing subsystem (microsecond) and slow supervision subsystem (167ms)
- Hardware-Level Fault Isolation: Independent clock, power supply, physical isolation for each module

3.7 Time Hierarchy & Tikhonov Separation Theorem

- Time Level Split (TLS0-TLS5): TLS0 (167ms, observer supervision), TLS4 (microsecond, electromagnetic field oscillation)

- Control Logic: Slow layer supervises fast layer, no interference with fast-layer internal evolution; boundary constraint and convergence acceptance only
- Tikhonov Theorem Support: Fast-layer self-stabilization + slow-layer rigid boundary guarantee global system stability

3.8 Noether's Theorem & Conservation Laws

- Time Translation Symmetry → Energy Conservation: No long-term system drift or divergence
- Space Translation Symmetry → Momentum Conservation: Stable energy distribution
- Rotation Symmetry (72° for five-element ring) → Circulation Conservation: Fixed energy conversion ratio between electric and magnetic fields
- Triple conservation laws: Ultimate guarantee of system stability

4. Implementation Framework of LINGXSEH Computer

4.1 Core Architecture

- **Electromagnetic Oscillation Core:** 2.45GHz resonant cavity, periodic electric-magnetic field conversion, self-compensation and self-correction oscillation unit
- **Digital Control & Computing Unit:** FPGA-implemented three CDEQ theorems, PWM modulation, ADC sampling, closed-loop control
- **State Visualization & Acceptance:** Black-and-white CRT display; snow screen (chaotic state), stable pattern (converged state)

4.2 Five-Layer Engineering Architecture

1. Physical Layer (TLS4): Electromagnetic constraint cavity, coil/electrode array, oscillation core, energy coupling interface
2. Electromagnetic Layer (TLS4): Oscillation unit array, phase stabilization, energy equalization, state acquisition
3. Logic Layer (L1): Field state recognition, logic conversion, synchronization de-jitter, local clipping, Q14 verification
4. Master Control & Convergence Layer (TLS0): 167ms timing, global boundary monitoring, convergence verification, central clipping scheduling
5. Application Layer: High-speed signal processing, device control, navigation calculation, scientific computing, AI acceleration

4.3 Five-Tier Progressive Methodology

1. Physical Intuition Layer: Electromagnetic oscillation pendulum analogy
2. Conservation Law Layer: Energy, momentum, angular momentum conservation
3. Quantitative Calculation Layer: Poynting vector quantitative description
4. Mathematical Specification Layer: Maxwell equations
5. Physical Law Layer: Noether's theorem (symmetry → conservation)

4.4 Closed-Loop Control Logic

- Power Injection: FPGA calculates five-element state, modulates magnetron power via PWM
- State Sensing: Five-probe array samples field intensity, Q14 quantified after ADC conversion
- Constraint Execution: Real-time nine-grid balance monitoring, gradient correction for deviation
- Convergence Judgment: 10 consecutive steps of cost function below threshold, raise convergence flag

4.5 Essential Differences from Traditional Computers

- Physical Carrier: Traditional (electron migration) vs. LINGXSEH (electromagnetic field evolution)
- Operation Rhythm: Traditional (clock synchronization) vs. LINGXSEH (natural field oscillation)
- Energy Consumption: Traditional (exponential growth with computing power) vs. LINGXSEH (linear growth)
- Stability Basis: Traditional (process, heat dissipation, error correction) vs. LINGXSEH (physical conservation laws)
- Scalability: Traditional (transistor stacking) vs. LINGXSEH (resonant cavity array parallelism)
- Convergence: Traditional (probabilistic) vs. LINGXSEH (mathematically guaranteed 10-step convergence)

5. Engineering Feasibility & Paradigm Significance

- Mature industrial components: Commercial microwave resonant cavity, FPGA, CRT display;< 1000 RMB
- Proven electromagnetic control technology: Medical, household, communication applications verify controllability
- Paradigm Revolution: Break free from silicon-based process limits, abandon EUV lithography dependence, realize computing power popularization via standard industrial processing
- Core Value: Reconstruct computing essence from "electron pushing" to "field evolution"

6. Core Legal & Physical Iron Laws

1. Electric and magnetic fields evolve subluminally in constrained space, serving as computing physical foundation
2. Poynting vector: Legal unified description of energy, momentum and computing power
3. Self-compensation/self-correction oscillation unit: Unique system timing benchmark
4. Field state conversion: Direct mapping to deterministic logic 0/1
5. Operation rhythm: Strictly corresponding to TLS4 microsecond-level oscillation
6. Slow-layer supervising fast-layer: Exclusive legal control mode
7. Four core guarantees: Boundary constraint, allowed evolution, out-of-bound control, result convergence
8. Tikhonov separation theorem: Ultimate mathematical stability guarantee
9. CENHE-LEX nine-grid constraint (Q14): Physical-level regular cage
10. Full compliance with CDEQ Time Charter V3.0-FINAL; three theorems TSA-certified with judicial validity

7. Conclusion

- Based on CDEQ framework, verify the dual nature of electromagnetic fields for energy transfer and computing
- Establish complete theoretical and implementation framework of LINGXSEH electromagnetic field computer
- Three CDEQ theorems, Poynting vector drive and Noether's conservation laws ensure deterministic computing
- Low-cost, normal temperature and pressure implementation feasible; break through silicon-based computing bottlenecks
- Future research directions: Custom resonant cavity design, multi-cavity parallel array, AI-dedicated architecture optimization

8. Validity & Burden of Proof

- Valid objections must satisfy at least one of the following:
 1. Overthrow Q14 fixed-point system or non-negativity premise
 2. Prove self-compensation oscillation unit cannot maintain periodic stability under nine-grid Q14 constraints
 3. Prove mathematical impossibility of 10-step convergence criterion
 4. Prove physical infeasibility of slow-layer supervising fast-layer architecture
 5. Overthrow any mathematical proof of three CDEQ theorems with rigorous counterevidence
- Failure to meet above conditions confirms system validity within CDEQ framework

9. Future Outlook

- Break silicon-based process monopoly: Millimeter-level resonant cavity processing, no EUV lithography required
- Linear scalability: Computing power increases linearly with resonant cavity quantity, no interconnection bandwidth bottleneck
- AI computing revolution: Eliminate parameter tuning "alchemy", realize deterministic, reproducible training via CDEQ rigid convergence
- Computing power popularization: Shift from high-end process monopoly to standard industrial mass production

10. References

Classical Physics & Mathematics

- [1] Maxwell, J.C. A Treatise on Electricity and Magnetism. Clarendon Press, 1873.
- [2] Noether, E. Invariante Variationsprobleme. Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse, 1918, pp. 235–257.
- [3] Tikhonov, A.N. Systems of differential equations containing small parameters in the derivatives. Matematicheskii Sbornik, 1952, 31(73): 575–586.
- [4] Jackson, J.D. Classical Electrodynamics (3rd ed.). Wiley, 1998.
- [5] Griffiths, D.J. Introduction to Electrodynamics (4th ed.). Cambridge University Press, 2017.

CDEQ Core Literature

- [6] Jiang Wenjia. Mathematical Proof of Penta-Cyclic-Clockwise-Topological-Theorem[Z/OL]. Zenodo, 2026. DOI:10.5281/zenodo.18988669.
- [7] Jiang Wenjia. Mathematical Proof of Central-Holding-Nonary-Constraint-Execution-Theorem[Z/OL]. Zenodo, 2026. DOI:10.5281/zenodo.19023440.
- [8] Jiang Wenjia. Mathematical Proof of Global-Convergence-Validation-Theorem[Z/OL]. Zenodo, 2026. DOI:10.5281/zenodo.19055969.
- [9] Jiang Wenjia. CDEQ V1.6: Proof of Legitimacy, Optimality of Deviation, and Uniqueness for the Distribution of Remainder 12 to the Corner Anchors[Z/OL]. Zenodo, 2026. DOI:10.5281/zenodo.19095712.

Algorithm & Architecture

- [10] Knuth, D.E. The Art of Computer Programming, Volume 1: Fundamental Algorithms (3rd ed.). Addison-Wesley, 1997.
- [11] Hennessy, J.L., Patterson, D.A. Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann, 2017.
- [12] Hamming, R.W. The Art of Doing Science and Engineering: Learning to Learn. CRC Press, 2020.

CDEQ Six Charters (Timestamp Descending)

- [13] Jiang Wenjia. CDEQ Foundational Charter, V3.0-FINAL, TSA-01-20260309579843370, 2026.
- [14] Jiang Wenjia. CDEQ Mathematical Charter, V3.2-FINAL, TSA-01-20260308577700362, 2026.
- [15] Jiang Wenjia. CDEQ Engineering Charter, V3.9-FINAL, TSA-01-20260308578910312, 2026.
- [16] Jiang Wenjia. CDEQ Global Coding Charter, V3.8-FINAL, TSA-01-20260308577343292, 2026.
- [17] Jiang Wenjia. CDEQ Time Charter, V3.0-FINAL, TSA-01-20260308577021982, 2026.
- [18] Jiang Wenjia. CDEQ Governance Charter, V1.0-FUSED, TSA-01-20260307550822631, 2026.

Intellectual Property Statement

All theoretical frameworks, formulas, algorithms, codes, and structural designs in this paper are the sole and original intellectual property of Jiang Wenjia. The complete manuscript has been legally registered and certified with a trusted TSA timestamp prior to submission. Any unauthorized use, plagiarism, misappropriation, or authorship falsification will result in immediate and full legal action.

This paper is the foundational charter of the CDEQ framework. All subsequent versions and derivative works must cite this original source.

A complete chain of evidence, including six foundational charters with trusted timestamps, predates this paper and confirms its sole authorship and originality. Any form of plagiarism or misappropriation can be easily identified and verified.

The six foundational charters serve as the unique verification code of originality. They are the cat at the door — only the true owner knows its name. No counterfeit can pass the check.

This is not a warning. This is a blade. If you reach for the fruit, expect to lose the hand.

Silicon Solidification Note

Full Verilog code implementing PCTT-CW, CENHE-LEX, and GCVT-10 is provided in Appendix A. The design uses zero conditional branches, pure combinational logic, and achieves SIL-4 safety readiness.

Appendix A:

```
``verilog
// =====
// CDEQ Triple Theorem Silicon Fixed Core
// OPEN Development Edition (Paper Appendix Version)
//
// Language: SystemVerilog (IEEE 1800-2012)
//           Compatible with Vivado, Quartus, Synopsys DC, VCS.
//
// Features:
//   - PCTT-CW, CENHE-LEX, GCVT-10
//   - Q14 fixed-point | History snapshot | Causal lock
//   - Clip bounded | Zero business IF
//   - 64-bit intermediate safe compute
//
// Synthesizable, ready for FPGA/ASIC implementation.
//
// IMPORTANT NOTES:
//   1. w_in must be a decaying chaotic weight (Q14, 0~16383)
//      e.g. W(t) = 16384 * max(0.05, 1 - sqrt(t/79)) for t=0..79
//      Constant input will NOT converge.
//   2. Recommended initial values:
//      p_init[4:0] = all 16'd8192 (0.5)
//      cenhe_init[1][1] = 16'd5460 (center)
//      other cenhe_init entries = 16'd1365 (approx 0.0833)
```

```

//      This ensures initial sums near target.
// =====

module cdeq_triple_core (
    input      clk,
    input      rst_n,
    input  [15:0] w_in,           // Chaotic weight W(t) (Q14, 0~16383)
    input  [15:0] p_init[4:0],   // PCTT-CW 5-element initial values
    input  [15:0] cenhe_init[3][3], // CENHE-LEX 3x3 grid initial values
    output reg  [15:0] p[4:0],    // PCTT-CW state output (reg -> always)
    output reg  [15:0] cenhe[3][3], // CENHE-LEX state output (reg -> always)
    output wire      converged    // GCVT-10 convergence flag (wire -> assign)
);

// -----
// 1. Global constants (fused, immutable)
// -----
localparam LINGBX_Q14_1P0    = 17'd16384; // 1.0 (17-bit to avoid sign ambiguity)
localparam LINGBX_Q14_HALF   = 16'd8192;  // 0.5
localparam LINGBX_PCTT_CW_S  = 16'd10124; // 0.618041
localparam LINGBX_PCTT_CW_C  = 16'd6258;  // 0.381958
localparam LINGBX_CLIP_MAX   = 16'd16383;
localparam LINGBX_CLIP_MIN   = 16'd0;
localparam LINGBX_GRAD_COEF  = 16'd1638;   // 0.1 * 16384
localparam LINGBX_GCVT_EPS   = 16'd1636;   // Convergence threshold
localparam LINGBX_GCVT_STEPS = 4'd10;      // GCVT-10: 10 consecutive steps

// -----
// 2. PCTT-CW dynamics (power generation)
// -----
reg [15:0] p_hist[4:0]; // History snapshot (reg -> always)

// Signed conversion for arithmetic
wire signed [15:0] p_hist_s[4:0];
assign p_hist_s[0] = $signed(p_hist[0]);
assign p_hist_s[1] = $signed(p_hist[1]);
assign p_hist_s[2] = $signed(p_hist[2]);
assign p_hist_s[3] = $signed(p_hist[3]);
assign p_hist_s[4] = $signed(p_hist[4]);

// Clockwise topology: predecessor tables (causality locked)
wire [2:0] prev[4:0], prev2[4:0];
assign prev[0] = 3'd4; prev[1] = 3'd0; prev[2] = 3'd1; prev[3] = 3'd2; prev[4] = 3'd3;
assign prev2[0] = 3'd3; prev2[1] = 3'd4; prev2[2] = 3'd0; prev2[3] = 3'd1; prev2[4] = 3'd2;

// History update: take snapshot at every clock
integer i;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin

```

```

        for (i = 0; i < 5; i = i + 1)
            p_hist[i] <= p_init[i];
    end else begin
        for (i = 0; i < 5; i = i + 1)
            p_hist[i] <= p[i];
        end
    end
end

// PCTT-CW update (strictly golden-ratio driven, using history only)
wire signed [63:0] delta_p[4:0];
wire signed [15:0] new_p[4:0];
generate
    genvar j;
    for (j = 0; j < 5; j = j + 1) begin : gen_p
        wire signed [63:0] term_s = $signed(LINGBX_PCTT_CW_S) * p_hist_s[prev[j]];
        wire signed [63:0] term_c = $signed(LINGBX_PCTT_CW_C) * p_hist_s[prev2[j]];
        wire signed [63:0] diff = term_s - term_c;          // Q28
        assign delta_p[j] = diff >>> 14;                  // shift to Q14
        assign new_p[j] = $signed(p_hist[j]) + delta_p[j];
    end
endgenerate

// Saturation (IF_LOCK_01) – reg driven by always
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (i = 0; i < 5; i = i + 1)
            p[i] <= p_init[i];
    end else begin
        for (i = 0; i < 5; i = i + 1) begin
            if (new_p[i] > $signed(LINGBX_CLIP_MAX))
                p[i] <= LINGBX_CLIP_MAX;
            else if (new_p[i] < $signed(LINGBX_CLIP_MIN))
                p[i] <= LINGBX_CLIP_MIN;
            else
                p[i] <= new_p[i];
        end
    end
end

end

// -----
// 3. CENHE-LEX constraint layer (structural equilibrium)
// -----
reg [15:0] cenhe_hist[3][3];
wire [16:0] R[2:0], C[2:0];    // 17-bit unsigned row/column sums

integer m, n;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (m = 0; m < 3; m = m + 1)

```

```

        for (n = 0; n < 3; n = n + 1)
            cenhe_hist[m][n] <= cenhe_init[m][n];
    end else begin
        for (m = 0; m < 3; m = m + 1)
            for (n = 0; n < 3; n = n + 1)
                cenhe_hist[m][n] <= cenhe[m][n];
            end
        end
    end

// Row sums & column sums (combinatorial, based on history)
assign R[0] = cenhe_hist[0][0] + cenhe_hist[0][1] + cenhe_hist[0][2];
assign R[1] = cenhe_hist[1][0] + cenhe_hist[1][1] + cenhe_hist[1][2];
assign R[2] = cenhe_hist[2][0] + cenhe_hist[2][1] + cenhe_hist[2][2];
assign C[0] = cenhe_hist[0][0] + cenhe_hist[1][0] + cenhe_hist[2][0];
assign C[1] = cenhe_hist[0][1] + cenhe_hist[1][1] + cenhe_hist[2][1];
assign C[2] = cenhe_hist[0][2] + cenhe_hist[1][2] + cenhe_hist[2][2];

// Gradient correction (only for edge-middle cells)
wire signed [15:0] grad[3][3];
generate
    for (m = 0; m < 3; m = m + 1) begin
        for (n = 0; n < 3; n = n + 1) begin : gen_grad
            wire is_edge_mid = ((m==0 && n==1) || (m==1 && n==0) ||
                                (m==1 && n==2) || (m==2 && n==1));
            // Extend to 34 bits to avoid any truncation warning
            wire signed [33:0] grad_raw = is_edge_mid ?
                ( $signed(LINGBX_GRAD_COEF) * (($signed(LINGBX_Q14_1P0) - $signed(R[m])) +
                                                  ($signed(LINGBX_Q14_1P0) - $signed(C[n]))) ) : 34'sd0;
            assign grad[m][n] = grad_raw >>> 14;
        end
    end
endgenerate

// Four-corner injection: (P_k - 8192) * scale * W(t)
wire signed [16:0] pk_centered[4];    // 17-bit signed to hold [-8192, 8191]
assign pk_centered[0] = $signed({1'b0, p_hist[0]}) - $signed(LINGBX_Q14_HALF);
assign pk_centered[1] = $signed({1'b0, p_hist[1]}) - $signed(LINGBX_Q14_HALF);
assign pk_centered[2] = $signed({1'b0, p_hist[2]}) - $signed(LINGBX_Q14_HALF);
assign pk_centered[3] = $signed({1'b0, p_hist[3]}) - $signed(LINGBX_Q14_HALF);

wire signed [15:0] scale[4];
assign scale[0] = 16'sd4369;    // 4/15
assign scale[1] = 16'sd2185;    // 2/15
assign scale[2] = 16'sd6554;    // 6/15
assign scale[3] = 16'sd8738;    // 8/15

wire signed [15:0] new_cenhe[3][3];
// Explicitly take lower 16 bits after shift to avoid width mismatch warnings
assign new_cenhe[0][0] = $signed(cenhe_hist[0][0]) +

```

```

        $signed( ((pk_centered[0] * scale[0] * $signed(w_in)) >>> 28)[15:0] );
assign new_cenhe[0][2] = $signed(cenhe_hist[0][2]) +
        $signed( ((pk_centered[1] * scale[1] * $signed(w_in)) >>> 28)[15:0] );
assign new_cenhe[2][2] = $signed(cenhe_hist[2][2]) +
        $signed( ((pk_centered[2] * scale[2] * $signed(w_in)) >>> 28)[15:0] );
assign new_cenhe[2][0] = $signed(cenhe_hist[2][0]) +
        $signed( ((pk_centered[3] * scale[3] * $signed(w_in)) >>> 28)[15:0] );

assign new_cenhe[0][1] = $signed(cenhe_hist[0][1]) + grad[0][1];
assign new_cenhe[1][0] = $signed(cenhe_hist[1][0]) + grad[1][0];
assign new_cenhe[1][2] = $signed(cenhe_hist[1][2]) + grad[1][2];
assign new_cenhe[2][1] = $signed(cenhe_hist[2][1]) + grad[2][1];

// Center anchor (engineering fixed value 5460)
assign new_cenhe[1][1] = 16'sd5460;

// Saturation for the 3x3 grid (reg driven by always)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (m = 0; m < 3; m = m + 1)
            for (n = 0; n < 3; n = n + 1)
                cenhe[m][n] <= cenhe_init[m][n];
    end else begin
        for (m = 0; m < 3; m = m + 1) begin
            for (n = 0; n < 3; n = n + 1) begin
                if (new_cenhe[m][n] > $signed(LINGBX_CLIP_MAX))
                    cenhe[m][n] <= LINGBX_CLIP_MAX;
                else if (new_cenhe[m][n] < $signed(LINGBX_CLIP_MIN))
                    cenhe[m][n] <= LINGBX_CLIP_MIN;
                else
                    cenhe[m][n] <= new_cenhe[m][n];
            end
        end
    end
end

end

// -----
// 4. GCVT-10 global convergence verification
// -----

reg [3:0] steady_cnt;
wire signed [63:0] v_pctt, v_cenhe;
wire signed [63:0] phi;

// V_PCTT (PCTT-CW deviation function)
wire signed [63:0] sum_sq = $signed(p_hist[0])*$signed(p_hist[0]) +
        $signed(p_hist[1])*$signed(p_hist[1]) +
        $signed(p_hist[2])*$signed(p_hist[2]) +
        $signed(p_hist[3])*$signed(p_hist[3]) +
        $signed(p_hist[4])*$signed(p_hist[4]);

```

```

wire signed [63:0] sum_cross1 = $signed(p_hist[0])*$signed(p_hist[4]) +
                                $signed(p_hist[1])*$signed(p_hist[0]) +
                                $signed(p_hist[2])*$signed(p_hist[1]) +
                                $signed(p_hist[3])*$signed(p_hist[2]) +
                                $signed(p_hist[4])*$signed(p_hist[3]);

wire signed [63:0] sum_cross2 = $signed(p_hist[0])*$signed(p_hist[3]) +
                                $signed(p_hist[1])*$signed(p_hist[4]) +
                                $signed(p_hist[2])*$signed(p_hist[0]) +
                                $signed(p_hist[3])*$signed(p_hist[1]) +
                                $signed(p_hist[4])*$signed(p_hist[2]);

wire signed [63:0] v_pctt_num = sum_sq -
                                ( ($signed(LINGBX_PCTT_CW_S) * sum_cross1) >>> 14 ) +
                                ( ($signed(LINGBX_PCTT_CW_C) * sum_cross2) >>> 14 );
assign v_pctt = v_pctt_num >>> 14;

// V_CENHE (CENHE-LEX deviation function, based on history snapshots)
wire signed [31:0] diff_row0 = $signed(R[0]) - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_row0 = diff_row0 * diff_row0;
wire signed [31:0] diff_row1 = $signed(R[1]) - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_row1 = diff_row1 * diff_row1;
wire signed [31:0] diff_row2 = $signed(R[2]) - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_row2 = diff_row2 * diff_row2;

wire signed [31:0] diff_col0 = $signed(C[0]) - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_col0 = diff_col0 * diff_col0;
wire signed [31:0] diff_col1 = $signed(C[1]) - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_col1 = diff_col1 * diff_col1;
wire signed [31:0] diff_col2 = $signed(C[2]) - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_col2 = diff_col2 * diff_col2;

wire signed [31:0] sum_diag0 = $signed(cenhe_hist[0][0]) +
                                $signed(cenhe_hist[1][1]) +
                                $signed(cenhe_hist[2][2]);
wire signed [31:0] diff_diag0 = sum_diag0 - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_diag0 = diff_diag0 * diff_diag0;

wire signed [31:0] sum_diag1 = $signed(cenhe_hist[0][2]) +
                                $signed(cenhe_hist[1][1]) +
                                $signed(cenhe_hist[2][0]);
wire signed [31:0] diff_diag1 = sum_diag1 - $signed(LINGBX_Q14_1P0);
wire signed [63:0] d_diag1 = diff_diag1 * diff_diag1;

wire signed [63:0] v_cenhe_num = d_row0 + d_row1 + d_row2 +
                                d_col0 + d_col1 + d_col2 +
                                d_diag0 + d_diag1;
assign v_cenhe = v_cenhe_num >>> 14;

```

```

assign phi = v_pctt + v_cenhe;

// Decision logic: only ?: operator, no conditional branch
wire step_ok = (phi < $signed(LINGBX_GCVT_EPS)) ? 1'b1 : 1'b0;

// Counter update (clear if step fails)
wire [3:0] inc = step_ok ? 4'd1 : 4'd0;
wire [3:0] next_cnt = (steady_cnt == LINGBX_GCVT_STEPS - 1) ? steady_cnt : steady_cnt + inc;
wire [3:0] cnt_en = step_ok ? next_cnt : 4'd0;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        steady_cnt <= 0;
    else
        steady_cnt <= cnt_en;
end

// Convergence flag (wire type, driven by assign) – "wire's dad is assign"
assign converged = (steady_cnt == LINGBX_GCVT_STEPS - 1 && step_ok);

endmodule

```